

Make `std::make_from_tuple` SFINAE friendly

Document #: P3738R1 [[Latest](#)] [[Status](#)]
Date: 2026-05-06
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply-to: Yihan Wang
<yronglin777@gmail.com>
Wenming Wang
<wenmingw@nvidia.com>
Jun Yang
<juney@nvidia.com>

Contents

- [1 Changelog](#)
 - [1.1 R1](#)
 - [1.2 R0](#)
- [2 Introduction](#)
- [3 Motivation](#)
- [4 Impact on the Standard](#)
- [5 Implementation Experience](#)
- [6 Proposed Wording](#)
- [7 Acknowledgements](#)
- [References](#)

§ 1 Changelog

§ 1.1 R1

- Make the original Mandates as Constraints.

§ 1.2 R0

Initial revision

§ 2 Introduction

This paper introduce constraints for `std::make_from_tuple` to make it SFINAE friendly.

§ 3 Motivation

[LWG3528] introduce constraints:

```
template<class T, class Tuple, size_t... I>
    requires is_constructible_v<T, decltype(get<I>(declval<Tuple>()))...>
constexpr T make-from-tuple-impl(Tuple&& t, index_sequence<I...>) {    //
    exposition only
    return T(get<I>(std::forward<Tuple>(t))...);
}
```

When someone write SFINAE code like the following to check whether T can constructed from a tuple, they may hit hard errors like “no matching function for call to `make-from-tuple-impl`” ([Compiler Explorer](#)).

```
template <typename T, typename Tuple, typename = void>
inline constexpr bool has_make_from_tuple = false;

template <typename T, typename Tuple>
inline constexpr bool has_make_from_tuple<
    T, Tuple,
    std::void_t<decltype(std::make_from_tuple<T>(std::declval<Tuple>
        ()))>> =
    true;
struct A {
    int a;
};
static_assert(!has_make_from_tuple<int *, std::tuple<A *>>);
```

Even If the effects are *Equivalent to* calling a constrained function, the constraints has not apply to `std::make_from_tuple`.

This is somehow unclear when the constraints are not literally specified with *Constraints* in the standard wording (16.3.2.4 [[structure.specifications](#)]). At least *Equivalent to* doesn't propagate every substitution failure in immediate context. In the case of `make-from-tuple-impl`, the constraints were introduced via a `requires`-clause but not literal *Constraints*. Some implementors believed the `requires`-clause should be treated same as *Constraints*, but this is not explicitly stated.

This proposal also suggests changing the *Mandates* “If `tuple_size_v<remove_reference_t<Tuple>>` is 1, then `reference_constructs_from_temporary_v<T, decltype(get<0>(declval<Tuple>()))>` is `false`” to a *Constraints*; this would yield similar benefits for the following example:

```
static_assert(!has_make_from_tuple<const int &, std::tuple<short>>);
```

§ 4 Impact on the Standard

This proposal is a pure library improvement for the current specification [N4944].

§ 5 Implementation Experience

I've partial implemented this improvement in `libc++`, `microsoft/STL`, `libstdc++`.

The constraints “If `tuple_size_v<remove_reference_t<Tuple>>` is 1, then `reference_constructs_from_temporary_v<T, decltype(get<0>(declval<Tuple>()))>` is `false`” was not implemented in these libraries.

§ 6 Proposed Wording

Modify section 22.4.6 22.4.6 [tuple.apply] as indicated:

```
template<class T, tuple-like Tuple>
constexpr T make_from_tuple(Tuple&& t);
```

- 2 *Mandates:* If `tuple_size_v<remove_reference_t<Tuple>>` is 1, then `reference_constructs_from_temporary_v<T, decltype(get<0>(declval<Tuple>()))>` is `false`.
- 2 Let `I` be the pack `0, 1, ..., (tuple_size_v<remove_reference_t<Tuple>> - 1)`.
- 3 *Constraints:*
 - `is_constructible_v<T, decltype(get<I>(declval<Tuple>()))...>` is `true`.
 - If `tuple_size_v<remove_reference_t<Tuple>>` is 1, then `reference_constructs_from_temporary_v<T, decltype(get<0>(declval<Tuple>()))>` is `false`.

Effects: Given the exposition-only function template:

```
namespace std {
    template<class T, tuple-like Tuple, size_t... I>
        requires is_constructible_v<T, decltype(get<I>(declval<Tuple>()))...>
        requires is_constructible_v<T, decltype(get<I>(declval<Tuple>()))...>
            &&
            tuple_size_v<std::remove_reference_t< Tuple>> == 1 &&
            !reference_constructs_from_temporary_v<T, decltype(get<0>(de-
            clval<Tuple>()))>
    constexpr T make-from-tuple-impl(Tuple&& t, index_sequence<I...>) { //
        exposition only
        return T(get<I>(std::forward<Tuple>(t))...);
    }
}
```

Equivalent to:

```
return make-from-tuple-impl<T>(
    std::forward<Tuple>(t),
```

```
        make_index_sequence<tuple_size_v<remove_reference_t<Tuple>>>>  
    {});
```

[*Note*: The type of `T` must be supplied as an explicit template parameter, as it cannot be deduced from the argument list. — *end note*]

§ 7 Acknowledgements

Thank you to Mark Hoemmen, Jiang An, Mark de Wever, Stephan T. Lavavej, Jonathan Wakely, Barry Revzin, Daniel Krügler, and everyone else who contributed to the discussions, and encouraged me to write this paper.

§ References

[LWG3528] Tim Song. `make_from_tuple` can perform (the equivalent of) a C-style cast.

<https://wg21.link/lwg3528>

[N4944] Thomas Köppe. 2023-03-22. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4944>